

ICS321 – Database Systems – Project #2

Campus Events Database



Mohammed Alfaraj – 202323090 – Section 1

Redha Alturaik – 202323010 – Section 3

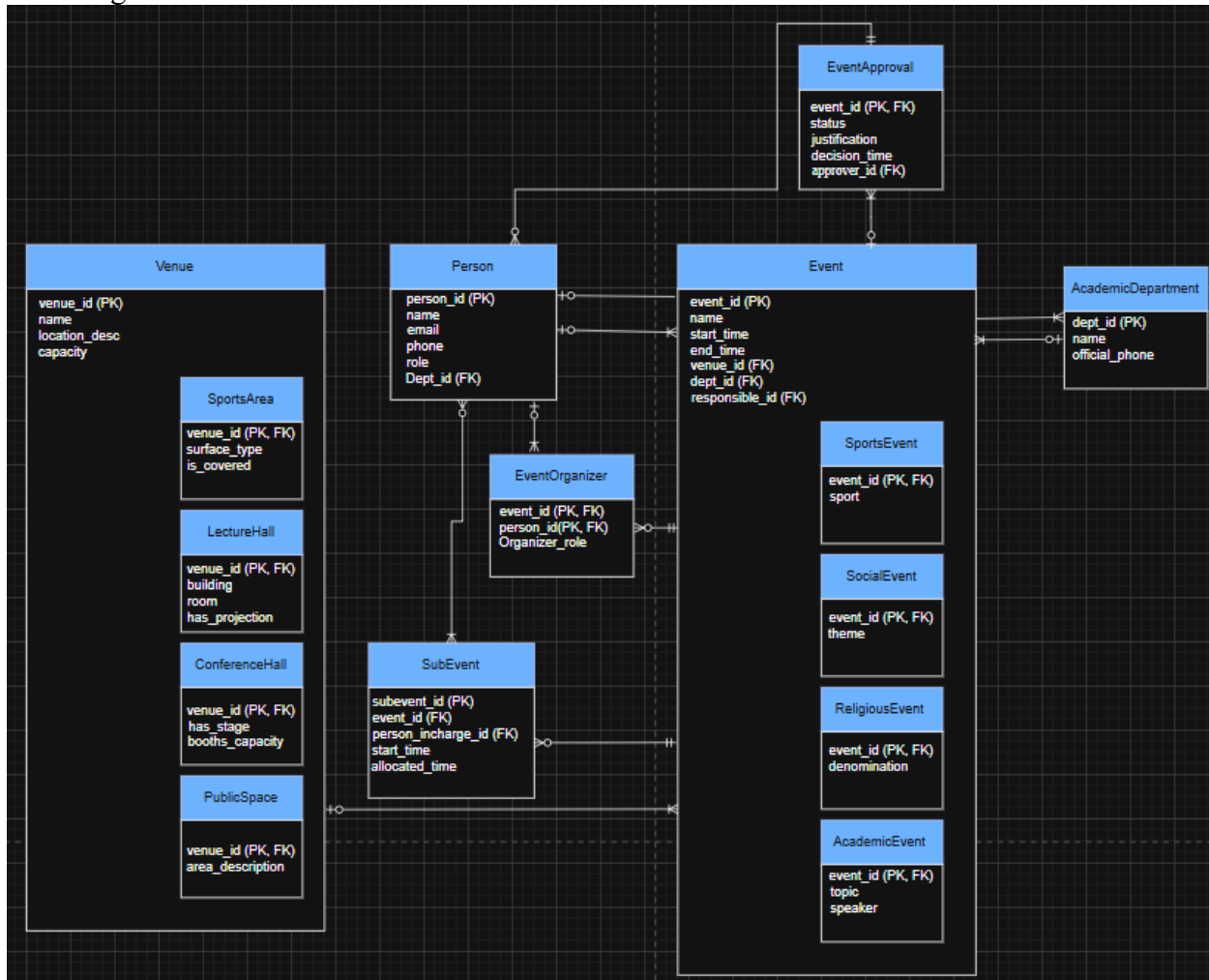
Group: Project M11

Date: 12/13/2025

Term 251

Design:

EER diagram:



Requirement 8:

```
Create event event_approval_notification
After UPDATE ON EventApproval
FOR each row when (new.status='Approved')
Begin
    -- send into a notification log
End;
```

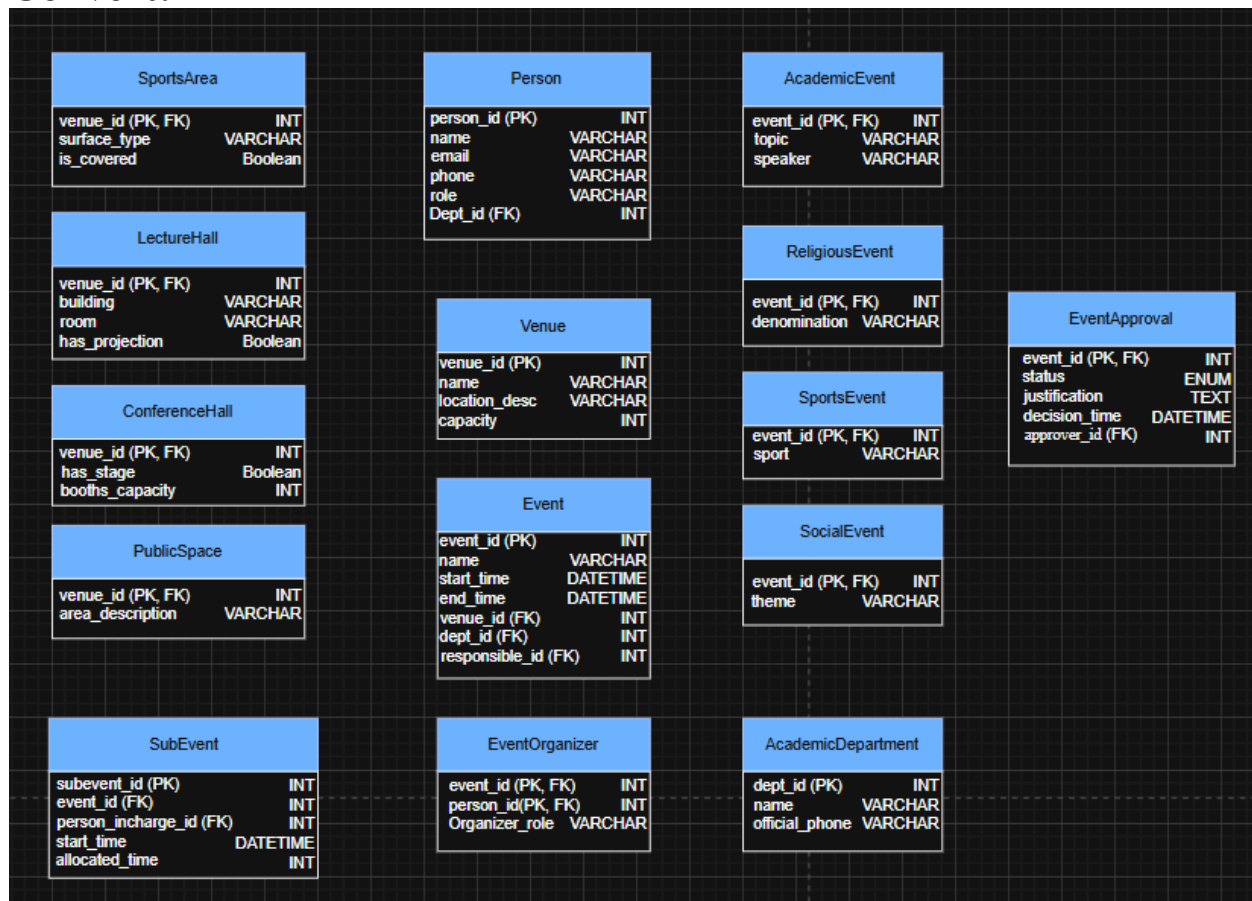
Requirement 9:

```
CHECK(end_time > start_time),
CHECK(TIMESTAMPDIFF(hour, start_time, end_time) <= 72),
CHECK(hour(start_time) BETWEEN 8 AND 23),
CHECK(hour(end_time) BETWEEN 8 AND 23)
```

Assumptions:

- Only students, staff, dependents, or faculty can create and organize events and subevents.
- The duration of events can start from 8 in the morning until 12 Midnight. Anytime before and after is not allowed. And they can last for 3 days
- Approvals start as pending and can be approved or rejected.
- Notification of event will be sent to maintenance, office services, and security departments.
- Events can have many subevents that can have different organizer, but the same main event organizer is still acceptable.
- A person may not have a department.

Convert:



AcademicDepartment(dept_id (PK), name, official_phone)

Person(person_id (PK), name, email, phone, role, dept_id (FK))

Venue(venue_id (PK), name, location_desc, capacity)

SportsArea(venue_id (PK, FK), surface_type, is_covered)

LectureHall(venue_id (PK, FK), building, room, has_projection)

ConferenceHall(venue_id (PK, FK), has_stage, booths_capacity)

PublicSpace(venue_id (PK, FK), area_description)

Event(event_id (PK), name, start_time, end_time, venue_id (FK), dept_id (FK), responsible_id (FK))

SportsEvent(event_id (PK, FK), sport)

SocialEvent(event_id (PK, FK), theme)

ReligiousEvent(event_id (PK, FK), denomination)

AcademicEvent(event_id (PK, FK), topic, speaker)

EventOrganizer(event_id (PK, FK), person_id(PK, FK), organizer_role)

EventApproval(event_id (PK, FK), status, justification, decision_time, approver_id (FK))

SubEvent(subevent_id (PK), event_id (FK), person_incharge_id (FK), start_time, allocated_time)

NotificationLog(notification_id (PK), event_id (FK), department, message, created_at)

All entries start off in unnormalized form.

All entries are in 1NF since all of them satisfy all conditions:

- The table has a primary key.
- Every non-key column depends on the primary key.
- The table cannot have duplicate rows.
- Every cell contains exactly one value.

All entities are in 2NF since either all non-key attributes rely on the whole composite key or only one primary key exists.

All entities are in 3NF since there are no transitive dependencies in tables.

All entities are in BCNF since all dependencies have a determinant that is a candidate key, meaning that all dependencies $X \rightarrow Y$, where X is a candidate key.

Tools and Resources Used

Tools:

- Draw.IO
- MySQL Server and Workbench 8.0

Resources:

- Zybook
- MySQL Documentation
- W3School

Tasks Done by Each Group Member

Tasks done by Mohammed Alfaraj:

- Logical Design
- BCNF conversion
- Create Tables
- Insert Statements

Tasks done by Redha Alturaik:

- EER model
- Insert Statements
- Stored Procedure
- Testing

Develop:

AcademicDepartment:

```
Create table AcademicDepartment(  
    dept_id INT primary key AUTO_INCREMENT,  
    name varchar(80) NOT NULL,  
    official_phone varchar(15)  
);
```

Person:

```
Create table Person (  
    person_id INT primary key AUTO_INCREMENT,  
    name varchar(30) NOT NULL,  
    email varchar(60) UNIQUE NOT NULL,  
    phone varchar(15),  
    role varchar(50) NOT NULL check(role IN ('student', 'faculty', 'staff', 'dependent')),  
    dept_id INT,  
    foreign key(dept_id) references AcademicDepartment(dept_id)  
);
```

Venue:

```
Create table Venue (  
    venue_id INT primary key AUTO_INCREMENT,  
    name varchar(80) NOT NULL,  
    location_desc varchar(200),  
    capacity INT  
);
```

SportsArea:

```
Create table SportsArea (  
    venue_id INT primary key,  
    surface_type varchar(60),  
    is_covered Boolean,  
    foreign key(venue_id) references Venue(venue_id)  
    on delete cascade  
);
```

LectureHall:

```
Create table LectureHall (  
    venue_id INT primary key,  
    building varchar(60),  
    room varchar(20),  
    has_projection Boolean,  
    foreign key(venue_id) references Venue(venue_id)  
    on delete cascade  
);
```

ConferenceHall:

```
Create table ConferenceHall (  
    venue_id INT primary key,  
    has_stage boolean,  
    booths_capacity INT,  
    foreign key(venue_id) references Venue(venue_id)  
    on delete cascade  
);
```

PublicSpace:

```
Create table PublicSpace (  
    venue_id INT primary key,  
    area_description varchar(80),  
    foreign key(venue_id) references Venue(venue_id)  
    on delete cascade  
);
```

Event:

```
Create table Event (  
    Event_id INT primary key AUTO_INCREMENT,  
    Name varchar(100) NOT NULL,  
    Start_time DATETIME NOT NULL,  
    End_time DATETIME NOT NULL,  
    Venue_id INT,  
    dept_id INT,  
    responsible_id INT,  
    foreign key(venue_id) references Venue(venue_id),  
    foreign key(dept_id) references AcademicDepartment (dept_id),  
    foreign key(responsible_id) references Person(person_id),  
    CHECK(end_time > start_time),
```

```
        CHECK(TIMESTAMPDIFF(hour, start_time, end_time) <= 72),
        CHECK(hour(start_time) BETWEEN 8 AND 23),
        CHECK(hour(end_time) BETWEEN 8 AND 23)
    );
```

SportsEvent:

```
Create table SportsEvent (
    event_id INT primary key,
    sport varchar(70),
    foreign key(event_id) references Event(event_id)
    on delete cascade
);
```

SocialEvent:

```
Create table SocialEvent (
    event_id INT primary key,
    theme varchar(70),
    foreign key(event_id) references Event(event_id)
    on delete cascade
);
```

ReligiousEvent:

```
Create table ReligiousEvent (
    event_id INT primary key,
    denomination varchar(70),
    foreign key(event_id) references Event(event_id)
    on delete cascade
);
```

AcademicEvent:

```
Create table AcademicEvent (
    event_id INT primary key,
    topic varchar(70),
    speaker varchar(40),
    foreign key(event_id) references Event(event_id)
    on delete cascade
);
```


EventOrganizer:

```
Create table EventOrganizer (  
    event_id INT,  
    person_id INT,  
    organizer_role varchar(40),  
    primary key(event_id, person_id),  
    foreign key(event_id) references Event(event_id)  
    on delete cascade,  
    foreign key(person_id) references Person(person_id)  
  
);
```

EventApproval:

```
Create table EventApproval (  
    event_id INT primary key,  
    status ENUM('pending', 'approved', 'rejected') DEFAULT 'pending',  
    justification TEXT,  
    decision_time DATETIME,  
    approver_id INT,  
    foreign key (event_id) references Event(event_id),  
    foreign key (approver_id) references Person(person_id)  
  
);
```

SubEvent:

```
Create table SubEvent (  
    subevent_id INT primary key AUTO_INCREMENT,  
    event_id INT,  
    person_incharge_id INT,  
    start_time DATETIME NOT NULL,  
    allocated_time INT NOT NULL,  
    foreign key(event_id) references Event(event_id)  
    on delete cascade,  
    foreign key(person_incharge_id) references Person(person_id)  
  
);
```

NotificationLog for trigger:

```
Create table NotificationLog (  
    notification_id INT primary key AUTO_INCREMENT,  
    event_id INT,  
    department varchar(50),  
    message TEXT,  
    created_at DATETIME default CURRENT_TIMESTAMP,  
    foreign key(event_id) references Event(event_id)
```

);

Trigger:

DELIMITER \$\$

Create trigger event_approval_notification

After UPDATE ON EventApproval

FOR each row

Begin

 If New.status = 'approved' AND (OLD.status is NULL OR OLD.status <> 'approved') then

 Insert into NotificationLog (event_id, department, message) Values

 (New.event_id, 'maintenance', concat('Event ', NEW.event_id, ' approved, maintenance has been notified')),

 (New.event_id, 'office_service', concat('Event ', NEW.event_id, ' approved, office service has been notified')),

 (New.event_id, 'security', concat('Event ', NEW.event_id, ' approved, security has been notified'));

End if;

End\$\$

DELIMITER;

View UpcomingEvents:

Create or replace view UpcomingEvents AS

Select e.event_id, e.name, e.start_time, e.end_time, v.name AS venue, d.name AS department, p.name AS responsible

From event e

Left join venue v ON e.venue_id = v.venue_id

Left join AcademicDepartment d ON e.dept_id = d.dept_id

Left join Person p ON e.responsible_id = p.person_id

Where e.start_time > now()

Order by e.start_time;

Populate:

AcademicDepartment:

Insert into AcademicDepartment (name, official_phone) values
('Physical Education', '2041'),
('Computer Science', '1047'),
('Student Affairs', '4913'),
('Social Studies', '8401'),
('Computer Engineering', '0612'),
('Mechanical Engineering', '4015'),
('Electrical Engineering', '3128');

Person:

Insert into Person (name, email, phone, role, dept_id) values
('Mohammed', 'Mohammed@kfupm.edu', '056-000-0001', 'staff', 1),
('Ali', 'Ali@kfupm.edu', '056-000-0002', 'student', 2),
('Hassan', 'Hassan@kfupm.edu', '056-000-0003', 'faculty', 3),
('Redha', 'Redha@kfupm.edu', '056-000-0004', 'dependent', 4),
('Faisal', 'Faisal@kfupm.edu', '056-000-0005', 'student', 5),
('Salah', 'Salah@kfupm.edu', '056-000-0006', 'staff', NULL),
('Omar', 'Omar@kfupm.edu', '056-000-0007', 'faculty', NULL);

Venue:

Insert into Venue (name, location_desc, capacity) values
('Main Field', 'Outdoor sports field', 250),
('North Field', 'Outdoor sports field', 200),
('South Field', 'Outdoor sports field', 200),
('East Field', 'Outdoor sports field', 200),
('West Field', 'Outdoor sports field', 200),
('Gym A', 'Indoor Gym', 50),
('Gym B', 'Indoor Gym', 50),
('Gym C', 'Indoor Court', 50),
('Conference Centre Room 1', 'Conference Building', 500),
('Conference Centre Room 2', 'Conference Building', 300),
('Conference Centre Room 3', 'Conference Building', 400),
('Stadium', 'University Stadium', 10000),
('Multipurpose Hall', 'Multipurpose Hall', 800),
('Seminar Room 1', 'Building 24', 100),
('Seminar Room 2', 'Building 6', 80),
('B63 Hall A', 'Building 63 – Hall A', 100),

('B63 Hall B', 'Building 63 – Hall B', 150),
('B22 Hall A', 'Building 22 – Hall A', 100),
('B22 Hall B', 'Building 22 – Hall B', 100),
('B22 Hall C', 'Building 22 – Hall C', 80);

SportsArea:

Insert into SportsArea (venue_id, surface_type, is_covered) values
(1, 'Grass', FALSE),
(8, 'Hardwood', TRUE),
(3, 'Grass', FALSE),
(6, 'Gym flooring', TRUE),
(7, 'Gym flooring', TRUE);

LectureHall:

Insert into LectureHall (venue_id, building, room, has_projection) values
(16, 'Building 63', 'A101', TRUE),
(17, 'Building 63', 'B201', TRUE),
(18, 'Building 22', 'A101', TRUE),
(19, 'Building 22', 'B201', TRUE),
(20, 'Building 22', 'C301', TRUE);

ConferenceHall:

Insert into ConferenceHall (venue_id, has_stage, booths_capacity) values
(9, TRUE, 500),
(10, TRUE, 300),
(11, TRUE, 400),
(13, TRUE, 800),
(14, False, 100);

PublicSpace:

Insert into PublicSpace (venue_id, area_description) values
(2, 'Large Field North'),
(4, 'Large Field East'),
(15, 'Medium Sized Seminar Room'),
(12, 'University Stadium'),
(8, 'Open Gym');

Event:

Insert into Event (name, start_time, end_time, venue_id, dept_id , responsible_id) values

('Football Competition', '2025-12-12 16:00:00', '2025-12-12 18:00:00', 12, 1, 1),
('Basketball Competition', '2025-12-13 16:00:00', '2025-12-13 18:00:00', 8, 1, 1),
('Volleyball Competition', '2025-12-14 16:00:00', '2025-12-14 18:00:00', 8, 1, 1),
('Tennis Competition', '2025-12-15 16:00:00', '2025-12-15 18:00:00', 8, 1, 1),
('Sprint Competition', '2025-12-16 16:00:00', '2025-12-16 18:00:00', 12, 1, 1),
('Social Gathering', '2025-12-10 13:00:00', '2025-12-10 18:00:00', 2, 3, 3),
('Student Festival', '2025-12-17 08:00:00', '2025-12-17 17:00:00', 4, 3, 3),
('Gaming Tournament', '2025-12-14 14:00:00', '2025-12-14 18:00:00', 9, 3, 3),
('Community Talk', '2025-12-20 10:00:00', '2025-12-20 12:00:00', 16, 3, 3),
('Meet and Greet', '2025-12-22 14:00:00', '2025-12-22 17:00:00', 14, 3, 3),
('Islamic Prayer', '2025-12-10 11:30:00', '2025-12-10 12:00:00', 5, 4, 4),
('Islamic Talk', '2025-12-10 13:00:00', '2025-12-10 15:00:00', 15, 4, 4),
('Islamic Ceremony A', '2025-12-10 14:00:00', '2025-12-10 16:00:00', 13, 4, 4),
('Islamic Ceremony B', '2025-12-11 14:00:00', '2025-12-11 16:00:00', 13, 4, 4),
('Islam Ceremony C', '2025-12-12 14:00:00', '2025-12-12 16:00:00', 13, 4, 4),
('Python101', '2025-12-16 19:00:00', '2025-12-16 21:00:00', 18, 2, 2),
('Essay Writing', '2025-12-17 19:00:00', '2025-12-17 21:00:00', 1, 4, 4),
('Problem Solving Skills', '2025-12-18 19:00:00', '2025-12-18 22:00:00', 11, 3, 3),
('Hackathon', '2025-12-19 18:00:00', '2025-12-19 20:00:00', 19, 2, 2),
('Quick Reading skills', '2025-12-20 19:00:00', '2025-12-20 21:00:00', 10, 4, 4);

SportsEvent:

Insert into SportsEvent (event_id, sport) values

(1, 'Football'),
(2, 'Basketball'),
(3, 'Volleyball'),
(4, 'Tennis'),
(5, 'Running');

SocialEvent:

Insert into SocialEvent (event_id, theme) values

(6, 'Social Talk'),
(7, 'Festival'),
(8, 'Gaming'),
(9, 'Community Talk'),
(10, 'Meet and Greet');

ReligiousEvent:

Insert into ReligiousEvent (event_id, denomination) values

(11, 'Prayer'),
(12, 'Sharing Islamic Knowledge'),
(13, 'Faith'),
(14, 'Faith'),
(15, 'Faith');

AcademicEvent:

Insert into AcademicEvent (event_id, topic, speaker) values
(16, 'Python Programming', 'Dr. Ali'),
(17, 'Writing an Essay', 'Dr. Redha'),
(18, 'Solving Critical Problems', 'Dr. Hassan'),
(19, 'Hackathon', 'Mr. Ali'),
(20, 'Quick Reading', 'Dr. Redha');

EventOrganizer:

Insert into EventOrganizer (event_id, person_id, organizer_role) values
(1, 1, 'staff'),
(16, 3, 'student'),
(6, 3, 'faculty'),
(4, 1, 'staff'),
(18, 3, 'faculty');

EventApproval:

Insert into EventApproval (event_id, status, justification, decision_time, approver_id) values
(2, 'approved', 'For Sport's Competition', NULL, 3),
(7, 'approved', 'For Social Event', NULL, 3),
(12, 'approved', 'For Religious Event', NULL, 3),
(17, 'approved', 'For Academic Purposes', NULL, 3),
(19, 'pending', 'For Academic Purposes', NULL, NULL);

SubEvent:

Insert into SubEvent (event_id, person_incharge_id, start_time, allocated_time) values
(6, 3, '2025-12-13 17:00:00', 30),
(8, 3, '2025-12-17 10:30:00', 90),
(10, 4, '2025-12-10 13:30:00', 30),
(16, 5, '2025-12-17 20:00:00', 60),
(17, 5, '2025-12-19 19:45:00', 15);

USE:

DELIMITER \$\$

Create Procedure setup_database()

Begin

-- Insert here all insert statements go here and views

End\$\$

DELIMITER;

CALL setup_database();

Test:

List all events with department, venue, and responsible person:

```
SELECT
    e.event_id,
    e.name      AS event_name,
    e.start_time,
    e.end_time,
    v.name      AS venue_name,
    d.name      AS department_name,
    p.name      AS responsible_person
FROM Event e
LEFT JOIN Venue v      ON e.venue_id = v.venue_id
LEFT JOIN AcademicDepartment d ON e.dept_id = d.dept_id
LEFT JOIN Person p     ON e.responsible_id = p.person_id
ORDER BY e.start_time;
```

Output:

	event_id	event_name	start_time	end_time	venue_name	department_name	responsible_person
▶	11	Islamic Prayer	2025-12-10 11:30:00	2025-12-10 12:00:00	West Field	Social Studies	Redha
	6	Social Gathering	2025-12-10 13:00:00	2025-12-10 18:00:00	North Field	Student Affairs	Hassan
	12	Islamic Talk	2025-12-10 13:00:00	2025-12-10 15:00:00	Seminar Room 2	Social Studies	Redha
	13	Islamic Ceremony A	2025-12-10 14:00:00	2025-12-10 16:00:00	Multipurpose Hall	Social Studies	Redha
	14	Islamic Ceremony B	2025-12-11 14:00:00	2025-12-11 16:00:00	Multipurpose Hall	Social Studies	Redha
	15	Islam Ceremony C	2025-12-12 14:00:00	2025-12-12 16:00:00	Multipurpose Hall	Social Studies	Redha
	1	Football Competition	2025-12-12 16:00:00	2025-12-12 18:00:00	Stadium	Physical Education	Mohammed
	2	Basketball Competition	2025-12-13 16:00:00	2025-12-13 18:00:00	Gym C	Physical Education	Mohammed
	8	Gaming Tournament	2025-12-14 14:00:00	2025-12-14 18:00:00	Conference Centre Room 1	Student Affairs	Hassan
	9	Volleyball Competition	2025-12-14 16:00:00	2025-12-14 18:00:00	Gym C	Physical Education	Mohammed

Find all sports events with their venue and surface details:

```
SELECT
    e.event_id,
    e.name      AS event_name,
    se.sport,
    v.name      AS venue_name,
    sa.surface_type,
    sa.is_covered
FROM SportsEvent se
JOIN Event e      ON se.event_id = e.event_id
JOIN Venue v      ON e.venue_id = v.venue_id
LEFT JOIN SportsArea sa ON v.venue_id = sa.venue_id
ORDER BY e.start_time;
```

	event_id	event_name	sport	venue_name	surface_type	is_covered
▶	1	Football Competition	Football	Stadium	NULL	NULL
	2	Basketball Competition	Basketball	Gym C	Hardwood	1
	3	Volleyball Competition	Volleyball	Gym C	Hardwood	1
	4	Tennis Competition	Tennis	Gym C	Hardwood	1
	5	Sprint Competition	Running	Stadium	NULL	NULL

Check the UpcomingEvents view:

```
SELECT
    event_id,
    name,
    start_time,
    end_time,
    venue,
    department,
    responsible
FROM UpcomingEvents
ORDER BY start_time;
```

Output:

	event_id	name	start_time	end_time	venue	department	responsible
▶	15	Islam Ceremony C	2025-12-12 14:00:00	2025-12-12 16:00:00	Multipurpose Hall	Social Studies	Redha
	1	Football Competition	2025-12-12 16:00:00	2025-12-12 18:00:00	Stadium	Physical Education	Mohammed
	2	Basketball Competition	2025-12-13 16:00:00	2025-12-13 18:00:00	Gym C	Physical Education	Mohammed
	8	Gaming Tournament	2025-12-14 14:00:00	2025-12-14 18:00:00	Conference Centre Room 1	Student Affairs	Hassan
	3	Volleyball Competition	2025-12-14 16:00:00	2025-12-14 18:00:00	Gym C	Physical Education	Mohammed
	4	Tennis Competition	2025-12-15 16:00:00	2025-12-15 18:00:00	Gym C	Physical Education	Mohammed
	5	Sprint Competition	2025-12-16 16:00:00	2025-12-16 18:00:00	Stadium	Physical Education	Mohammed
	16	Python101	2025-12-16 19:00:00	2025-12-16 21:00:00	B22 Hall A	Computer Science	Ali
	7	Student Festival	2025-12-17 08:00:00	2025-12-17 17:00:00	East Field	Student Affairs	Hassan
	17	Feesu Writing	2025-12-17 19:00:00	2025-12-17 21:00:00	Main Field	Social Studies	Redha

Events per department and type (academic / sports / social / religious):

```
SELECT
    d.name AS department_name,
    SUM(CASE WHEN ae.event_id IS NOT NULL THEN 1 ELSE 0 END) AS
academic_events,
    SUM(CASE WHEN se.event_id IS NOT NULL THEN 1 ELSE 0 END) AS sports_events,
    SUM(CASE WHEN so.event_id IS NOT NULL THEN 1 ELSE 0 END) AS social_events,
    SUM(CASE WHEN re.event_id IS NOT NULL THEN 1 ELSE 0 END) AS religious_events
FROM Event e
LEFT JOIN AcademicDepartment d ON e.dept_id = d.dept_id
LEFT JOIN AcademicEvent ae ON e.event_id = ae.event_id
LEFT JOIN SportsEvent se ON e.event_id = se.event_id
LEFT JOIN SocialEvent so ON e.event_id = so.event_id
```



```
LEFT JOIN ReligiousEvent re ON e.event_id = re.event_id
GROUP BY d.name
ORDER BY d.name;
```

Output:

	department_name	academic_events	sports_events	social_events	religious_events
►	Computer Science	2	0	0	0
	Physical Education	0	5	0	0
	Social Studies	2	0	0	5
	Student Affairs	1	0	5	0